第 1 章練習解答　導論

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS01-E01　共享機器的強制邊界

1. CPU不可永久獨占／改特權狀態；不可任意讀寫Alice RAM；不可任意讀寫她的file。三者相關但各有強制位置。
2. CPU用受保護timer/mode與kernel scheduler；RAM用MMU/PTE權限與OS維護映射；file操作透過受保護kernel入口、可信credentials與ACL/capability。檔案權限不是硬體直接理解pathname，硬體先保護執行檢查的kernel與metadata。
3. 全域共享clipboard/file可提升方便卻增加authority與資料外洩路徑；可改成明確授予指定物件／操作rights。GUI按鈕隱藏不能強制API。

檢查。 每個邊界列enforcement point；三問逐項對應。

OS01-E02　事件、mode與context

1. timer外部非同步；system call由目前指令同步發起。DMA完成狀態與CPU發現完成是不同事件，可由poll或interrupt通知。
2. kernel被timer打斷不必有user→kernel transition，也不必換process；cache-hit read通常user→kernel→user共2mode transitions，可回同thread而0process switch。若另外發生排程則另計。
3. 可以poll完成狀態；仍需正確cache／memory ordering及buffer lifetime。poll頻率、事件率、偵測latency與CPU成本決定取捨，不能把DMA等同每block必有一interrupt。

檢查。 來源分類不能代替實際切換計數。

OS01-E03　hash反例與生命期

1. bucket1為[5,9,13]；13需3比較，17也比3次後不存在。n個同餘keys可達O(n)，平均O(1)需hash／負載等前提。
2. 維持平衡與排序可提供O(log n)最壞搜尋及range查詢；若只查exact key，是否較合適仍依成本與同步，不以名稱決定。
3. 移出可達list不代表所有已取得reference都消失。立刻free／reuse可能use-after-free；需鎖、引用計數或滿足其讀者協定的grace period後回收。允許舊值不等於允許失效記憶體。

檢查。 查詢成本與生命期分開。

第 1 章練習解答　導論 （續）

OS01-E04　兩工作的I/O重疊

1. A CPU0–2，IO2–8，CPU8–10；B10–14；CPU8/14=4/7，總14。
2. A0–2，B2–6，idle6–8，A8–10；IO仍2–8，總10，CPU8/10=0.8，節省的4是B與IO重疊。
3. 沒有ready B、共享IO排隊、切換／同步overhead或memory壓力會改模型。單CPU沒有同時執行兩段CPU，總CPU工作仍8。

檢查。 列出idle，不只寫利用率。

OS01-E05　cache與NUMA的分母

1. 額外penalty模型1+0.1×50=6；miss總50模型0.9×1+0.1×50=5.9。輸入契約不同，不是兩式互相矛盾。
2. 0.8×100+0.2×180=116ns；0.2×100+0.8×180=164ns。
3. 不能；還要知道實際memory stall佔基準時間比例、重疊／cache／bandwidth與其餘工作。單一子系統成本不自動等於總程式時間。

檢查。 保留單位與條件。

OS01-E06　相同VA不等於相同資料

1. VPN=1，offset0x234=564；P PA=0x7000+0x234=0x7234；Q PA=0x9234。
2. P權限允許；Q為permission fault，不應無條件配新頁把禁止寫當作可補缺頁。具體COW政策須有另設標記及條件，本題無。
3. 不能；仍存在mapping/reference時可能讀到已重用別人資料。須確定舊引用／TLB／I/O等不再使用，再交free pool。

檢查。 駐留、權限與共享分開。

OS01-E07　timer與公平性的前提

1. 固定輪轉無更高priority插入且服務可完成，剛錯過者最多等(n−1)q+n c。若n改變或不可搶占段無界，此界不適用。
2. 每q工作加c切換，比例0.1/4.1=1/41≈2.44%；不是實測所有context switch成本。
3. 否。handler不終止時timer不能使執行回到有進度的scheduler；還需interrupt可服務、kernel bounded blocking及無永久飢餓的選取規則。

檢查。 硬體回收機制不代替排程策略。

第 1 章練習解答　導論 （續）

OS01-E08　遺失更新與BST範圍

1. T1load0，T2load0，T1store1，T2store1，結果1。單CPU可在步驟間切換，同樣成立。原子RMW／適用mutex可改成不可分割更新規格。
2. 12雖比parent5大，卻在root10的left subtree而12>10。按12>10往right20再往left空，會找不到存在的12；必須滿足全部子樹區間。
3. 1bit/block，共2^20bits÷8=2^17B=128KiB。這是bitmap邏輯容量，不是單次實體IO只有1bit。

檢查。 局部比較不是一般不變量。

OS01-E09　deadline與平台邊界

1. A0–1、B1–3、idle3–4、A4–5、idle5–6、B6–8、A8–9、idle9–12。A三次完成1/5/9各早於4/8/12；B3/8早於6/12。lcm12且無剩餘，接著可重複。
2. 不能；U=1/4+2/6=7/12只列需求率，不防任意不公平scheduler空轉／選錯。可行性、所選演算法保證與實際模型前提需分別證明。
3. VM通常各有guest kernel與VMM邊界；一般container共享host kernel，以namespace／資源限制等提供隔離視圖。不能把兩者同稱各有獨立OS kernel。

檢查。 時間正確性不是平均快。

OS01-E10　API、版本與來源可用性

1. 不足。OSI定義還要求授權允許相應散布與衍生等權利；source available不自動等同open source，逐component看實際license。
2. 不能；需固定版本、架構、configuration並看API可允許結果與實作，再用一般不變量及針對條件的觀察核對。一次成功不排除合法錯誤／競爭／故障分支。
3. 成功可能只到volatile cache；需定義file／metadata／device cache flush與完成次序、裝置遵約、必要名稱持久化及失效模型。只看到return success不足。

檢查。 不把證據種類互相替代。

第 1 章練習解答　導論 （續）

原創驗算程式附錄: intro_models.py

```
"""Standalone OS1 model: exact arithmetic and explicit sequential events."""
from fractions import Fraction
from itertools import permutations


def average_cache(hit=Fraction(9,10),lookup=1,extra=50):
    if not 0<=hit<=1 or lookup<0 or extra<0:raise ValueError('invalid cost')
    return lookup+(1-hit)*extra


def numa(local_fraction):
    if not 0<=local_fraction<=1:raise ValueError('ratio')
    return local_fraction*100+(1-local_fraction)*180


def translate(va,pages,page_size=4096,write=False):
    if va<0 or page_size<=0:raise ValueError('address')
    vpn,offset=divmod(va,page_size)
    if vpn not in pages:raise KeyError('not present')
    frame,writable=pages[vpn]
    if write and not writable:raise PermissionError('write fault')
    return frame*page_size+offset


def two_jobs():
    serial=[('A',0,2),('idle',2,8),('A',8,10),('B',10,14)]
    overlap=[('A',0,2),('B',2,6),('idle',6,8),('A',8,10)]
    util=lambda rows:Fraction(sum(end-start for who,start,end in rows if
who!='idle'),rows[-1][2])
    return
{'serial':serial,'overlap':overlap,'serial_utilization':util(serial),'overlap_utilization':ut
il(overlap),'io':('A',2,8)}


def lost_updates():
    """All orderings preserving each thread's load/add/store order.
    +1 is folded into store of the saved load; no memory-model claim.
    """
    events=['L1','S1','L2','S2'];outcomes={}
    for order in permutations(events):
        if order.index('L1')>order.index('S1') or
order.index('L2')>order.index('S2'):continue
        memory=0;registers={}
        for event in order:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 1 章練習解答　導論 （續）

```
            op,thread=event
            if op=='L':registers[thread]=memory
            else:memory=registers[thread]+1
        outcomes[order]=memory
    return outcomes


def bst_valid(node,lo=None,hi=None):
    if node is None:return True
    key,left,right=node
    if (lo is not None and key<=lo) or (hi is not None and key>=hi):return False
    return bst_valid(left,lo,key) and bst_valid(right,key,hi)


def edf_periodic(tasks,horizon):
    """Unit-tick preemptive model: (C, period, relative deadline), releases at0.
    No overhead/blocking; jobs may coexist. Tie by release then task id.
    """
    if horizon<0 or any(c<=0 or t<=0 or d<=0 for c,t,d in tasks):raise ValueError('task')
    jobs=[];timeline=[];completions=[]
    for now in range(horizon):
        for task,(cost,period,deadline) in enumerate(tasks):
            if
now%period==0:jobs.append({'task':task,'release':now,'deadline':now+deadline,'remaining':cost
})
        candidates=[job for job in jobs if job['remaining']>0]
        if not candidates:timeline.append(None);continue
        job=min(candidates,key=lambda j:(j['deadline'],j['release'],j['task']))
        job['remaining']-=1;timeline.append(job['task'])
        if
job['remaining']==0:completions.append((job['task'],job['release'],now+1,job['deadline']))
    return timeline,completions,jobs


if __name__=='__main__':
    print('cache cycles',average_cache())
    print('NUMA ns',numa(Fraction(4,5)),numa(Fraction(1,5)))
    print(two_jobs())
    print('PA',hex(translate(0x1234,{1:(7,True)})))
    print('counter outcomes',set(lost_updates().values()))
    print('EDF',edf_periodic([(1,4,4),(2,6,6)],12)[:2])
```

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study